

Minimum Eigen Optimizer Solve Function "list index out of range"

<https://github.com/qiskit-community/qiskit-optimization/issues/532>

ROW 119

Minimum Eigen Optimizer Solve Function "list index out of range" #532

[New issue](#)[Closed](#)

OuCheng-Fu opened on Jun 24, 2023

...

Environment

- Qiskit Optimization version: 0.5.0
- Python version: 3.9
- Operating system: Linux

What is happening?

What is the current behavior?

I try to solve a QUBO problem by QAOA, and implement it on Qiskit by this code

#Implement the reduction of variables mentioned in the Adrew Lucas Paper

```
mdl_red = Model(name="TSP-Variable Reduction")
```

```
m = n
```

```
y = {(j, l): mdl_red.binary_var(name=f"y_{j}_{l}") for j in range(m) for l in range(m) if j!=0 and l!=0}
```

```
tsp_func_red = mdl_red.sum(
```

```
1.0 * y[(j, l)] * y[(w, (l + 1) % m)]
```

```
for j in range(m)
```

```
for w in range(m)
```

```
for l in range(m)
```

```
if j != w and j!=0 and l!=0 and w!=0 and (l+1)%m != 0)
```

```
mdl_red.minimize(tsp_func_red + y[(1, 1)] + y[(1, 5)] + y[(2, 1)] + y[(2, 5)] + y[(3, 1)] + y[(3, 5)] + y[(4, 1)] +  
y[(4, 5)] + y[(5, 1)] + y[(5, 5)])
```

```
for j in range(1,m):
```

```
mdl_red.add_constraint(mdl_red.sum(y[(j, l)] for l in range(1,m)) == 1)
```

```
for l in range(1,m):
```

```
mdl_red.add_constraint(mdl_red.sum(y[(j, l)] for j in range(1,m)) == 1)
```

```
qp_red = from_docplex_mp(mdl_red)
```

```
print(qp_red.prettyprint())
```

```
#Construct Hamiltonian from TSP module
```

```
#Output QUBO Hamiltonian
```

```
from qiskit_optimization.converters import QuadraticProgramToQubo
```

```
qp2qubo_red = QuadraticProgramToQubo()
```

```
qubo_red = qp2qubo_red.convert(qp_red)
```

```
qubitOp_red, offset = qubo_red.to_ising()
```

```
print("Offset:", offset)
```

```
print("Ising Hamiltonian:")
```

```
print(str(qubitOp_red))
```

```
from qiskit.utils import algorithm_globals
```

```
from qiskit_algorithms.minimum_eigensolvers import QAOA, NumPyMinimumEigensolver
```

```
from qiskit_algorithms.optimizers import COBYLA
```

```
from qiskit.primitives import Sampler
```

```
from qiskit_optimization.algorithms import (
```

```
MinimumEigenOptimizer,
```

```
RecursiveMinimumEigenOptimizer,
```

```
SolutionSample,
```

```
OptimizationResultStatus,
```

```
)
```

```
from qiskit_optimization import QuadraticProgram
```

```
from qiskit.visualization import plot_histogram
```

```
from typing import List, Tuple
```

```
algorithm_globals.random_seed = 10598
```

```
qaoa_mes = QAOA(sampler=Sampler(), optimizer=COBYLA(), initial_point=[0.0, 0.0])
```

```
exact_mes = NumPyMinimumEigensolver()
```

```
qaoa = MinimumEigenOptimizer(qaoa_mes) # using QAOA
```

```
exact_2 = MinimumEigenOptimizer(exact_mes) # using the exact classical numpy minimum eigen solver
```

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#)[Subscribe](#)

You're not receiving notifications from this thread.

Participants



The bolded font code has the result

```
objective function value: 6.0
variable values: y_1_1=0.0, y_1_2=0.0, y_1_3=0.0, y_1_4=0.0, y_1_5=1.0, y_2_1=0.0, y_2_2=0.0, y_2_3=1.0, y_2_4=0.0, y_2_5=0.0,
y_3_1=1.0, y_3_2=0.0, y_3_3=0.0, y_3_4=0.0, y_3_5=0.0, y_4_1=0.0, y_4_2=0.0, y_4_3=0.0, y_4_4=1.0, y_4_5=0.0, y_5_1=0.0,
y_5_2=1.0, y_5_3=0.0, y_5_4=0.0, y_5_5=0.0
status: SUCCESS
```

```
However when I implement the code
qaoa_result_red = qaoa.solve(qubo_red)
print(qaoa_result_red)
```

It resulted in the error

```
IndexError Traceback (most recent call last)
Cell In[21], line 1
----> 1 qaoa_result_red = qaoa.solve(qubo_red)
      2 print(qaoa_result_red.prettyprint())
```

```
File ~/conda/envs/Qiskit/lib/python3.9/site-packages/qiskit_optimization/algorithms/minimum_eigen_optimizer.py:227, in
MinimumEigenOptimizer.solve(self, problem)
    224 # construct operator and offset
    225 operator, offset = problem._to_ising()
--> 227 return self.solve_internal(operator, offset, problem, problem)
```

```
File ~/conda/envs/Qiskit/lib/python3.9/site-packages/qiskit_optimization/algorithms/minimum_eigen_optimizer.py:271, in
MinimumEigenOptimizer._solve_internal(self, operator, offset, converted_problem, original_problem)
    260 return MinimumEigenOptimizationResult(
    261 x=None,
    262 fval=None,
    (...)
    267 min_eigen_solver_result=eigen_result,
    268 )
    270 # translate result back to integers and eventually maximization
--> 271 samples, best_raw = self._interpret_samples(original_problem, raw_samples, self._converters)
    272 return cast(
    273 MinimumEigenOptimizationResult,
    274 self._interpret(
    (...)
    282 ),
    283 )
```

```
File ~/conda/envs/Qiskit/lib/python3.9/site-packages/qiskit_optimization/algorithms/optimization_algorithm.py:522, in
OptimizationAlgorithm._interpret_samples(cls, problem, raw_samples, converters)
    516 samples.append(SolutionSample(x, fval, probability, status))
    518 sorted_samples = sorted(
    519 samples,
    520 key=lambda v: (v.status.value, problem.objective.sense.value * v.fval),
    521 )
--> 522 best_raw = raw_samples[index[tuple(sorted_samples[0].x)]]
    523 return sorted_samples, best_raw
```

IndexError: list index out of range

I tried to search the source code from

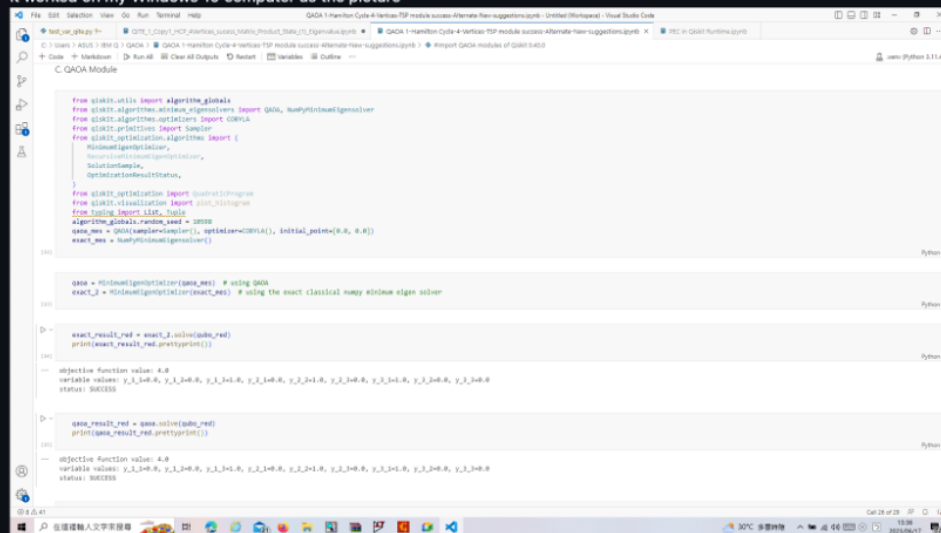
https://qiskit.org/ecosystem/optimization/_modules/qiskit_optimization/algorithms/minimum_eigen_optimizer.html#MinimumEigenOptimizer.solve

I found the code from the source, and the error comes from
samples, best_raw = self._interpret_samples(original_problem, raw_samples, self._converters)
but I can't find the source of function _interpret_samples()
Where is that function? What is the problem?

Moreover, I have tried it again on the example of 4 vertices with the same code,

```
qaoa_result_red = qaoa.solve(qubo_red)
print(qaoa_result_red)
```

it worked on my Windows 10 computer as the picture



```
from qiskit.utils import algorithm_globals
from qiskit.algorithms.minimum_eigensolvers import QAOA, NumpyMinimumEigensolver
from qiskit.algorithms.optimizers import COBYLA
from qiskit.primitives import Sampler
from qiskit.optimization.algorithms import (
    MinimumEigensolver,
    RecursionMinimumEigensolver,
    SolutionSample,
    OptimizationResultStatus,
)
from qiskit.optimization import QuadraticProgram
from qiskit.visualization import plot_histogram
from typing import List, Tuple
algorithm_globals.random_seed = 10598
qaoa_mes = QAOASamplerSampler(), optimizer=COBYLA(), initial_points=[0.0, 0.0]
exact_mes = NumpyMinimumEigensolver()

qaoa = MinimumEigensolver(qaoa_mes) # using QAOA
exact_mes = MinimumEigensolver(exact_mes) # using the exact classical numpy minimum eigensolver

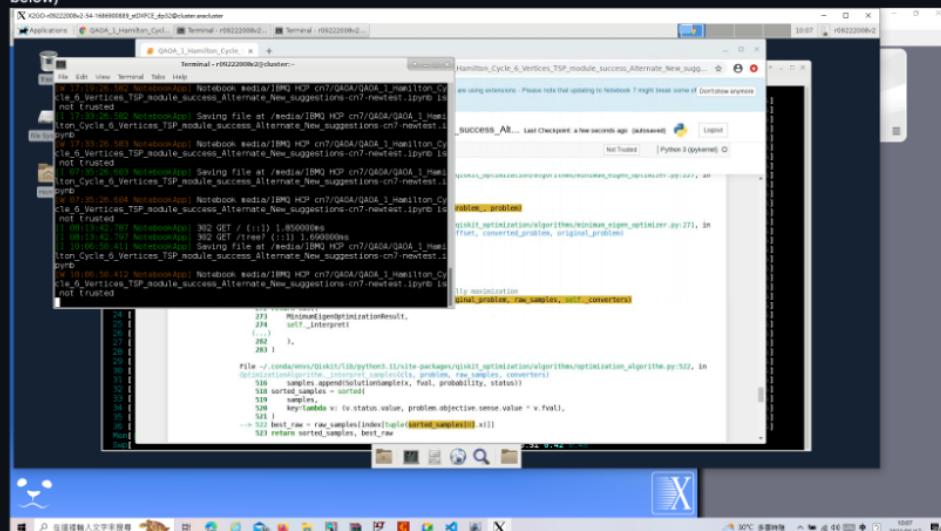
exact_result_mes = exact_mes.solve(qaoa_mes)
print(exact_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS

qaoa_result_mes = qaoa.solve(qaoa_mes)
print(qaoa_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS
```

However, when I try the same code on 6-vertex graph on a larger computer with 1.47TB memory, it ran for 15 hours (see picture below)



```
from qiskit.utils import algorithm_globals
from qiskit.algorithms.minimum_eigensolvers import QAOA, NumpyMinimumEigensolver
from qiskit.algorithms.optimizers import COBYLA
from qiskit.primitives import Sampler
from qiskit.optimization.algorithms import (
    MinimumEigensolver,
    RecursionMinimumEigensolver,
    SolutionSample,
    OptimizationResultStatus,
)
from qiskit.optimization import QuadraticProgram
from qiskit.visualization import plot_histogram
from typing import List, Tuple
algorithm_globals.random_seed = 10598
qaoa_mes = QAOASamplerSampler(), optimizer=COBYLA(), initial_points=[0.0, 0.0]
exact_mes = NumpyMinimumEigensolver()

qaoa = MinimumEigensolver(qaoa_mes) # using QAOA
exact_mes = MinimumEigensolver(exact_mes) # using the exact classical numpy minimum eigensolver

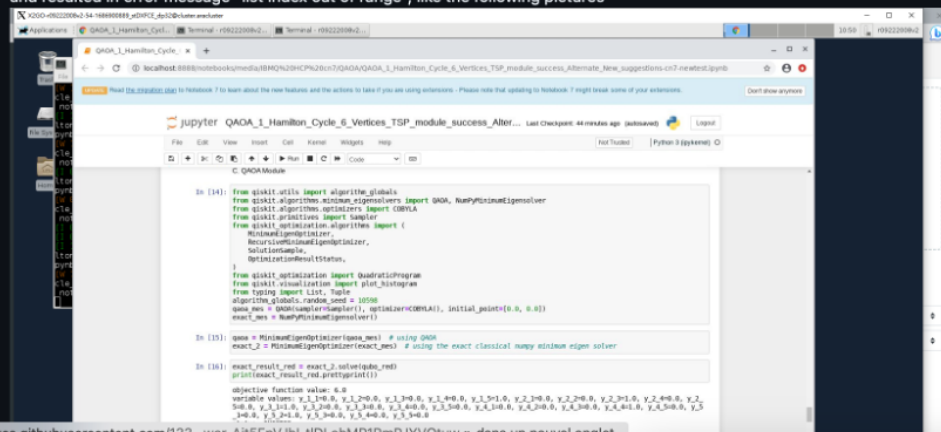
exact_result_mes = exact_mes.solve(qaoa_mes)
print(exact_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS

qaoa_result_mes = qaoa.solve(qaoa_mes)
print(qaoa_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS
```

and resulted in error message "list index out of range", like the following pictures



```
from qiskit.utils import algorithm_globals
from qiskit.algorithms.minimum_eigensolvers import QAOA, NumpyMinimumEigensolver
from qiskit.algorithms.optimizers import COBYLA
from qiskit.primitives import Sampler
from qiskit.optimization.algorithms import (
    MinimumEigensolver,
    RecursionMinimumEigensolver,
    SolutionSample,
    OptimizationResultStatus,
)
from qiskit.optimization import QuadraticProgram
from qiskit.visualization import plot_histogram
from typing import List, Tuple
algorithm_globals.random_seed = 10598
qaoa_mes = QAOASamplerSampler(), optimizer=COBYLA(), initial_points=[0.0, 0.0]
exact_mes = NumpyMinimumEigensolver()

qaoa = MinimumEigensolver(qaoa_mes) # using QAOA
exact_mes = MinimumEigensolver(exact_mes) # using the exact classical numpy minimum eigensolver

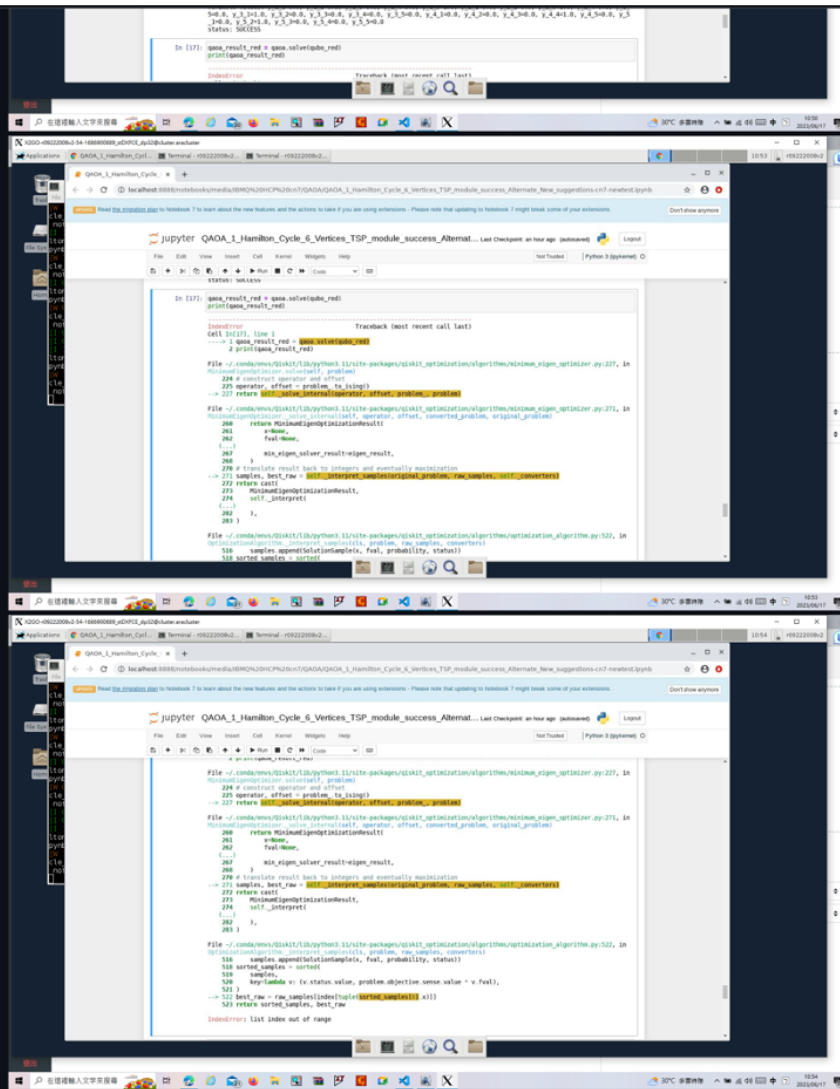
exact_result_mes = exact_mes.solve(qaoa_mes)
print(exact_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS

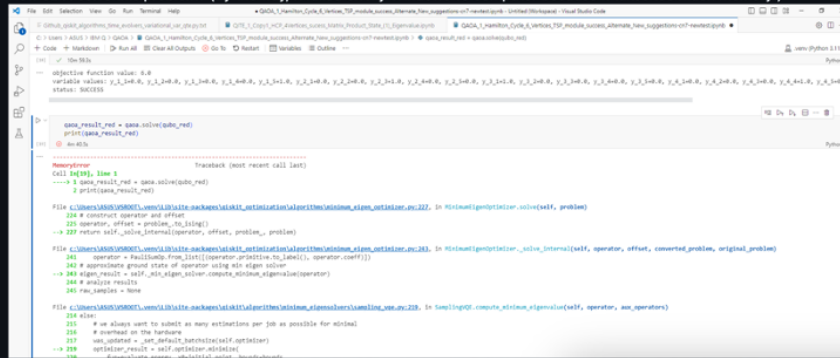
qaoa_result_mes = qaoa.solve(qaoa_mes)
print(qaoa_result_mes.prettyprint())

# objective function value: 4.0
# variable values: y_1,0=0.0, y_1,1=0.0, y_1,2=0.0, y_1,3=0.0, y_1,4=0.0, y_1,5=0.0, y_1,6=0.0, y_1,7=0.0, y_1,8=0.0, y_1,9=0.0
# status: SUCCESS
```

private-user-images.githubusercontent.com/133...wor-AH5FvVJh-iDI-ahMP1RmPJYV0tWw dans un nouvel onolet



How can I solve this problem? (By the way, it failed on my Windows 10 computer because it takes too much memory)



Any suggestions?

Where can I find the source code of the function? And what is the real issue?



OuCheng-Fu mentioned this on Jun 24, 2023

🔗 [Minimum Eigen Optimizer Solve Function "list index out of range" Qiskit/qiskit-tutorials#1468](#)



t-imamichi on Jun 26, 2023

Collaborator ...

This problem contains 25 binary variables (i.e., 25 qubits). So, the reference implementation `Sampler` is not sufficient to even obtain a single shot. You need to select an appropriate backend for 25 qubits such real device or Aer Sampler. In case of Aer Sampler, you may still require a lot of memory.



OuCheng-Fu on Jun 26, 2023

Author ...

This problem contains 25 binary variables (i.e., 25 qubits). So, the reference implementation `Sampler` is not sufficient to even obtain a single shot. You need to select an appropriate backend for 25 qubits such real device or Aer Sampler. In case of Aer Sampler, you may still require a lot of memory.

Thank you for reply. I have read the documentation

https://qiskit.org/documentation/tutorials/algorithms/05_qaoa.html

and its source code. How should I choose another Aer sampler (it seems that the code runs on Aer sampler, not real device)?



t-imamichi on Jun 26, 2023 · edited by t-imamichi

Edits ... Collaborator ...

It's not Aer's sampler.

The tutorial imports `from qiskit.primitives import Sampler`. It's Terra's reference primitive for small circuits.

See https://qiskit.org/ecosystem/aer/stubs/qiskit_aer.primitives.Sampler.html for Aer Sampler.

Note that it may still require a lot of memory and time for 25 qubits.



OuCheng-Fu on Jun 26, 2023

Author ...

It's not Aer's sampler. The tutorial imports `from qiskit.primitives import Sampler`. It's Terra's reference primitive for small circuits. See https://qiskit.org/ecosystem/aer/stubs/qiskit_aer.primitives.Sampler.html for Aer Sampler. Note that it may still require a lot of memory and time for 25 qubits.

Thank you. I will try it again by altering the code.



OuCheng-Fu on Jun 26, 2023

Author ...

It's not Aer's sampler. The tutorial imports `from qiskit.primitives import Sampler`. It's Terra's reference primitive for small circuits. See https://qiskit.org/ecosystem/aer/stubs/qiskit_aer.primitives.Sampler.html for Aer Sampler. Note that it may still require a lot of memory and time for 25 qubits.

I implemented it again by changing the sampler to

`from qiskit_aer.primitives import Sampler`

it succeeded, took 25 minutes to reproduce the exact result and 6.5 minutes to overcome where used to be the error.

Moreover, it took 16 minutes to calculate minimum eigenvalue, like the pictures below



Go

[illegible][illegible]